

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1205	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	25
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	P Dinesh Reddy	Reg.No.:	192525399
Department:		Date:	

Code of Conduct:

/ P Dinesh Reddy/192525399 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

Addition and Subtraction of Signed 8-bit Integers (2's Complement)

Given:

- $A = +45$
- $B = -23$

Step 1: Convert to 8-bit Binary

+45

$$45_{10} = 00101101_2$$

-23

First convert +23:

$$23_{10} = 00010111_2$$

Take 2's complement:

- 1's complement = 11101000
- Add 1 = **11101001**

Therefore,

$$-23 = \mathbf{11101001}$$

A. Addition (+45 + (-23))

00101101

+ 11101001

00010110

Carry out = 1 (discard in 8-bit arithmetic)

Result:

$$00010110 = +22$$

Verification

$$45 + (-23) = \mathbf{22}$$

Correct.

Overflow Detection

Overflow occurs only when:

- Positive + Positive $\rightarrow$ Negative

- Negative + Negative $\rightarrow$ Positive

Here,

- One positive
- One negative

Therefore,

No Overflow

Processor Interpretation

MSB = 0

Hence result is positive.

Final Answer:

+22

B. Subtraction (+45 - (-23))

Subtracting a negative equals addition.

45 - (-23)

=45 +23

Binary:

+23

00010111

Now,

00101101

+00010111

01000100

Decimal

$$01000100 = 68$$

Overflow Check

Operands:

Positive + Positive

Result:

Positive

Within range

-128 to +127

Therefore,

No Overflow

Processor Overflow Detection

The processor compares carry into MSB and carry out of MSB.

Overflow occurs if

Carry into MSB $\neq$ Carry out of MSB

or

$$V = C7 \text{ XOR } C8$$

If $V = 1$

Overflow

Else

No Overflow

Final Results

Operation	Result	Overflow
$45 + (-23)$	22	No
$45 - (-23)$	68	No

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry lookahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

Carry Lookahead Adder (CLA)

Suppose

$$A = 1101$$

$$B = 1011$$

Step 1: Generate (G) and Propagate (P)

Formula

$$G_i = A_i \times B_i$$

$$P_i = A_i \oplus B_i$$

Bit	A	B	G	P
0	1	1	1	0
1	0	1	0	1
2	1	0	0	1
3	1	1	1	0

Step 2: Carry Equations

Assume

$$C_0 = 0$$

$$C1 = G0 + P0C0$$

$$=1+0=1$$

$$C2 = G1 + P1G0 + P1P0C0$$

$$=0+1\times1+0$$

$$=1$$

$$C3 = G2 + P2G1 + P2P1G0 + P2P1P0C0$$

$$=0+0+1\times1\times1$$

$$=1$$

$$C4 = G3 + P3G2 + P3P2G1 + P3P2P1G0$$

$$=1$$

All carries are calculated simultaneously.

Sum Calculation

$$Si = Pi \text{ XOR } Ci$$

Each sum is obtained immediately after carry computation.

Ripple Carry Delay

Each bit waits for previous carry.

For 4 bits:

Delay

$$T = 4 \times t_{\text{carry}}$$

CLA Delay

Carries are generated simultaneously.

Approximate delay

$$T = t_{\text{generate}} + t_{\text{logic}}$$

Independent of ripple propagation.

Why CLA is Faster

- Parallel carry generation
- No waiting for previous stage
- Suitable for high-speed processors
- Reduces execution time

Comparison

Ripple Carry

Carry propagates sequentially

Slow

Simple hardware

Used in small circuits

Carry Lookahead

Carry computed simultaneously

Very Fast

More hardware

Used in CPUs

3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and $+5$. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

Booth's Algorithm ($-6 \times +5$)

Given

Multiplicand (M) $= -6$

Multiplier (Q) $= +5$

Using 4 bits

$M = 1010$

$-M = 0110$

$Q = 0101$

$A = 0000$

$Q-1 = 0$

Initial

$A = 0000$

$Q = 0101$

$Q-1 = 0$

Cycle 1

$Q_0 Q-1$

10

Subtract M

$A = A - M$

$A = 0110$

Arithmetic Shift

$A = 0011$

Q=0010

Q-1=1

Cycle 2

01

Add M

A=0011+1010

=1101

Shift

A=1110

Q=1001

Q-1=0

Cycle 3

10

Subtract

Shift

Continue.

Cycle 4

Repeat until 4 cycles complete.

Final Product

11100010

Decimal

−30

Verification

$-6 \times 5 = -30$

Correct.

Role of Registers

Accumulator (A)

Stores partial product.

Multiplier (Q)

Contains multiplier.

Booth Bit (Q−1)

Determines operation.

Arithmetic Right Shift

Maintains sign bit.

Ensures signed multiplication.

Advantages

- Handles positive and negative numbers directly.
- Consecutive 1's require only one addition and one subtraction.
- Reduces arithmetic operations.
- Faster than ordinary shift-add multiplication.

4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

Restoring and Non-Restoring Division ($13 \div 3$)

Binary

Dividend = 1101

Divisor = 0011

Expected answer

Quotient = 4

Remainder = 1

A. Restoring Division

Initial

A = 0000

Q = 1101

M = 0011

Step 1

Shift AQ

Subtract M

Negative

Restore

Quotient bit=0

Step 2

Shift

Subtract

Positive

Quotient=1

Step 3

Repeat

Step 4

Repeat

Final

Q=0100

A=0001

Answer

Quotient=4

Remainder=1

Non-Restoring Division

Initial

A=0000

Q=1101

Instead of restoring,
If remainder negative,
Next cycle perform addition.

If positive,
Perform subtraction.

Final correction

If remainder negative

$A = A + M$

Final

$Q = 0100$

$A = 0001$

Comparison

Restoring

Restores after negative remainder

More arithmetic operations

Slower

Higher hardware activity

Simple control

Non-Restoring

No restoration

Fewer operations

Faster

Less hardware activity

Slightly complex control

Why Non-Restoring is Preferred

- Fewer additions.
- No restore operation.
- Faster execution.

- Better processor utilization.
- Widely used in modern processors.

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

IEEE 754 Floating Point Representation

Given

-13.25

Step 1: Convert to Binary

Integer

$13 = 1101$

Fraction

$0.25 = 0.01$

Therefore

13.25

$= 1101.01_2$

Normalize

1.10101×2^3

Single Precision (32-bit)

Sign

Negative

$S = 1$

Exponent

Bias

127

Exponent

$3 + 127 = 130$

Binary

10000010

Mantissa

Remove leading 1

101010000000000000000000

Final Representation

Sign	Exponent	Mantissa
1	10000010	101010000000000000000000

Double Precision (64-bit)

Bias

1023

Exponent

1026

Binary

10000000010

Mantissa

1010100

Final

Field	Bits
Sign	1
Exponent	11
Mantissa	52

Floating Point Addition

Example

13.25

+

2.5

Step 1

Convert both into normalized form.

Step 2

Compare exponents.

Step 3

Shift smaller mantissa.

Step 4

Add mantissas.

Step 5

Normalize result.

Example

10.1110

↓

1.01110×2^4

Step 6

Round according to IEEE-754 (typically **round to nearest, ties to even**).

Precision Loss

Occurs because:

- Mantissa has finite bits.
- Some decimal fractions cannot be represented exactly in binary (e.g., 0.1).
- Smaller operands may lose least significant bits when aligned to a much larger exponent.
- Rounding introduces small approximation errors.

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation